

# Package ‘xtifedml’

September 30, 2026

**Type** Package

**Title** Double Machine Learning for Static Panel Models with Interactive Fixed Effects

**Version** 0.1.4

**Date** 2026-09-22

**Maintainer** Annalivia Polselli <apolSELLI.econ@gmail.com>

**Description** Implements partially linear panel regression (PLPR) models with interactive fixed effects, high-dimensional confounding variables, and an exogenous treatment variable within the double machine learning framework. Estimates the structural parameter (treatment effect) in static panel data models with interactive fixed effects using the approach established in Chen et al. (2026) <doi:10.48550/arXiv.2608.01137>. Builds on the object-oriented package 'DoubleML' (Bach et al., 2024) <doi:10.18637/jss.v108.i03> and 'xtDML' (Polselli, 2025) <doi:10.48550/arXiv.2512.15965>, using the 'mlr3' ecosystem.

**License** GPL-2 | GPL-3

**Encoding** UTF-8

**Depends** R (>= 4.1.0)

**Imports** R6 (>= 2.4.1), data.table (>= 1.12.8), mlr3 (>= 1.3.0), mlr3tuning (>= 1.5.0), mlr3learners (>= 0.13.0), mlr3misc (>= 0.19.0), mytnorm, utils, clusterGeneration, readstata13, magrittr, dplyr (>= 1.1.0), stats, MLmetrics, checkmate

**Suggests** rpart, bbotk (>= 1.8.0), testthat (>= 3.0.0), paradox, mlr3pipelines, ranger, xgboost, glmnet

**Config/roxygen2/version** 8.1.0

**Config/testthat/edition** 3

**NeedsCompilation** no

**Author** Binzhi Chen [aut] (ORCID: <<https://orcid.org/0000-0002-5094-7740>>),  
Annalivia Polselli [aut, cre] (ORCID:  
<<https://orcid.org/0009-0002-7579-7926>>)

**Repository** CRAN

**Date/Publication** 2026-09-30 12:10:02 UTC

## Contents

|                            |           |
|----------------------------|-----------|
| make_plpr_data . . . . .   | 2         |
| xtifedml . . . . .         | 3         |
| xtifedml_data . . . . .    | 9         |
| xtifedml_data_df . . . . . | 12        |
| xtifedml_plr . . . . .     | 13        |
| <b>Index</b>               | <b>19</b> |

---

|                |                                                                                                            |
|----------------|------------------------------------------------------------------------------------------------------------|
| make_plpr_data | <i>Data generating process for partially linear panel regression models with interactive fixed effects</i> |
|----------------|------------------------------------------------------------------------------------------------------------|

---

## Description

Generates data from a partially linear regression model for panel data with fixed effects from Binzhi et al. (2026).

The data generating process is defined as

$$Y_{it} = \theta D_{it} + l_0(X_{it}) + \lambda'_i f_t + U_{it}, D_{it} = m_0(X_{it}) + \lambda'_i f_t + V_{it},$$

where  $U_{it} \sim \mathcal{N}(0, 1)$ ,  $V_{it} \sim \mathcal{N}(0, 1)$ ,  $\lambda_i \sim \mathcal{N}(0, 2)$ ,  $f_t \sim \mathcal{N}(0, 2)$ .

The covariates are distributed as  $X_{it,p} \sim \mathcal{N}(1, 5) + \lambda'_i f_t$ , where  $p$  is the number of covariates.

The nuisance functions are generated as follows. For `dgp = "linear"`,

$$m_0(X_{it}) = a_1 X_{it,1} + a_2 X_{it,3},$$

$$g_0(X_{it}) = b_1 X_{it,1} + b_2 X_{it,3}.$$

For `dgp = "smooth"`,

$$m_0(X_{it}) = a_1 \max\{X_{it,1}, 0\} + a_2 |X_{it,3}|,$$

$$g_0(X_{it}) = b_1 [X_{it,1} \times 1\{X_{it,3} > 0\}] + b_2 [X_{it,3} \times X_{it,3} > 0],$$

For `dgp = "discouNTinuous"`,

$$m_0(X_{it}) = a_1 [X_{it,1} \times 1(X_{it,1} > 0)] + a_2 [X_{it,1} \times X_{it,3}],$$

$$g_0(X_{it}) = b_1 [X_{it,1} \times X_{it,3}] + b_2 [X_{it,3} \times 1(X_{it,3} > 0)].$$

where  $a_1 = b_2 = 0.25$  and  $a_2 = b_1 = 0.5$ .

## Usage

```
make_plpr_data(
  n_obs = 500,
  t_per = 10,
  dim_x = 20,
  theta = 1,
  r = 2,
  rho = 0.6,
  dgp = NULL
)
```

**Arguments**

|       |                                                                                                              |
|-------|--------------------------------------------------------------------------------------------------------------|
| n_obs | (integer(1))<br>The number of subjects (cross-sectional units) to simulate. Default 500.                     |
| t_per | (integer(1))<br>The number of time periods to simulate. Default 10.                                          |
| dim_x | (integer(1))<br>The number of covariates. Default 20.                                                        |
| theta | (numeric(1))<br>The value of the causal parameter. Default 1.                                                |
| r     | (numeric(1))<br>Number of factors. Default 2.                                                                |
| rho   | (numeric(1))<br>Parameter governing the correlation between the IFE and the included variables. Default 0.6. |
| dgp   | (character(1))<br>The type of dataset to generate. Default NULL.                                             |

**Value**

A data object.

**Examples**

```
df = make_plpr_data(n_obs = 500, t_per = 10, dim_x = 20, theta = 1, r=2, dgp = "discontinuous")
```

---

xtifedml

---

*Abstract Class xtifedml*


---

**Description**

Abstract base class that cannot be initialized directly.

Implements partially linear panel regression (PLPR) models with high-dimensional confounding variables and an exogenous treatment variable within the double machine learning framework. Estimates the structural parameter (treatment effect) in static panel data models with interactive fixed effects, using the approach of Binzhi et al. (2026).

Builds on the object-oriented architecture of DoubleML (Bach et al., 2024) and xtdml (Polselli, 2025), using the 'mlr3' ecosystem and the 'R6' package.

**Format**

[R6::R6Class](#) object.

**Active bindings**

`all_coef_theta` (matrix())  
 Estimates of the causal parameter(s) "theta" for the `n_rep` different sample splits after calling `fit()`.

`all_dml1_coef_theta` (array())  
 Estimates of the causal parameter(s) "theta" for the `n_rep` different sample splits after calling `fit()` with `dml_procedure = "dml1"`.

`all_se_theta` (matrix())  
 Standard errors of the causal parameter(s) "theta" for the `n_rep` different sample splits after calling `fit()`.

`all_model_rmse` (matrix())  
 Model root-mean-squared-error.

`apply_cross_fitting` (logical(1))  
 Indicates whether cross-fitting should be applied. Default is TRUE.

`coef_theta` (numeric())  
 Estimates for the causal parameter(s) "theta" after calling `fit()`.

`data` (data.table)  
 Data object.

`dml_procedure` (character(1))  
 A character() ("dml1" or "dml2") specifying the double machine learning algorithm. Default is "dml2".

`draw_sample_splitting` (logical(1))  
 Indicates whether the sample splitting should be drawn during initialization of the object. Default is TRUE.

`learner` (named list())  
 The machine learners for the nuisance functions.

`n_folds` (integer(1))  
 Number of folds. Default is 5.

`n_rep` (integer(1))  
 Number of repetitions for the sample splitting. Default is 1.

`params` (named list())  
 The hyperparameters of the learners.

`psi_theta` (array())  
 Value of the score function  $\psi(W; \theta_0, \eta_0) = -\psi_a(W; \eta_0)\theta_0 + \psi_b(W; \eta_0)$  after calling `fit()`.

`psi_theta_a` (array())  
 Value of the score function component  $\psi_a(W; \eta_0)$  after calling `fit()`.

`psi_theta_b` (array())  
 Value of the score function component  $\psi_b(W; \eta_0)$  after calling `fit()`.

`res_y` (array())  
 Residual of output equation

`res_d` (array())  
 Residual of treatment equation

`predictions` (array())  
Predictions of the nuisance models after calling `fit(store_predictions=TRUE)`.

`targets` (array())  
Targets of the nuisance models after calling `fit(store_predictions=TRUE)`.

`rmse` (array())  
The root-mean-squared-errors of the nuisance parameters

`all_model_mse` (array())  
Collection of all mean-squared-errors of the model

`model_rmse` (array())  
The root-mean-squared-errors of the model

`models` (array())  
The fitted nuisance models after calling `fit(store_models=TRUE)`.

`pval_theta` (numeric())  
p-values for the causal parameter(s) "theta" after calling `fit()`.

`score` (character(1))  
A character(1) specifying the score function among "orth-P0", "orth-IV". Default is "orth-P0".

`se_theta` (numeric())  
Standard errors for the causal parameter(s) "theta" after calling `fit()`.

`smpls` (list())  
The partition used for cross-fitting.

`smpls_cluster` (list())  
The partition used for cross-fitting. `smpl` is at cluster-var

`t_stat_theta` (numeric())  
t-statistics for the causal parameter(s) "theta" after calling `fit()`.

`tuning_res_theta` (named list())  
Results from hyperparameter tuning.

## Methods

### Public methods:

- `xtifedml$new()`
- `xtifedml$print()`
- `xtifedml$fit()`
- `xtifedml$split_samples()`
- `xtifedml$tune()`
- `xtifedml$summary()`
- `xtifedml$confint()`
- `xtifedml$learner_names()`
- `xtifedml$params_names()`
- `xtifedml$set_ml_nuisance_params()`
- `xtifedml$get_params()`
- `xtifedml$clone()`

`xtifedml$new()`: DML with IFE is an abstract class that can't be initialized.

*Usage:*

```
xtifedml$new()
```

`xtifedml$print()`: Print 'DML with IFE' objects.

*Usage:*

```
xtifedml$print()
```

`xtifedml$fit()`: Estimate DML models with IFE.

*Usage:*

```
xtifedml$fit(store_predictions = FALSE, store_models = FALSE)
```

*Arguments:*

`store_predictions` (logical(1))

Indicates whether the predictions for the nuisance functions should be stored in field `predictions`.  
Default is FALSE.

`store_models` (logical(1))

Indicates whether the fitted models for the nuisance functions should be stored in field `models` if you want to analyze the models or extract information like variable importance.  
Default is FALSE.

*Returns:* self

`xtifedml$split_samples()`: Draw sample splitting for Double ML models with IFE.

The samples are drawn according to the attributes `n_folds`, `n_rep` and `apply_cross_fitting`.

*Usage:*

```
xtifedml$split_samples()
```

*Returns:* self

`xtifedml$tune()`: Hyperparameter-tuning for DML models with IFE.

The hyperparameter-tuning is performed using the tuning methods provided in the [mlr3tuning](#) package. For more information on tuning in [mlr3](#), we refer to the section on parameter tuning in the [mlr3 book](#).

*Usage:*

```
xtifedml$tune(
  param_set,
  tune_settings = list(n_folds_tune = 5, rsmp_tune = mlr3::rsmp("cv", folds = 5), measure
    = NULL, terminator = mlr3tuning::trm("evals", n_evals = 20), tuner =
    mlr3tuning::tnr("grid_search", resolution = 5)),
  tune_on_folds = FALSE
)
```

*Arguments:*

`param_set` (named list())

A named list with a parameter grid for each nuisance model/learner (see method `learner_names()`).  
The parameter grid must be an object of class [ParamSet](#).

tune\_settings (named list())

A named list() with arguments passed to the hyperparameter-tuning with **mlr3tuning** to set up **TuningInstance** objects. tune\_settings has entries

- terminator (**Terminator**)  
A **Terminator** object. Specification of terminator is required to perform tuning.
- tuner (**Tuner**)  
A **Tuner** object created with **tnr()**, which defines the optimization algorithm (e.g., **tnr("grid\_search", resolution = 10)** or **tnr("random\_search")**). Any tuner-specific arguments, such as resolution for "grid\_search", must be supplied directly to **tnr()** when constructing this object. If not specified by the user, default is **tnr("grid\_search", resolution = 5)**.
- rsmp\_tune (**Resampling** or **character(1)**)  
A **Resampling** object (recommended) or option passed to **rsmp()** to initialize a **Resampling** for parameter tuning in **mlr3**. If not specified by the user, default is set to "cv" (cross-validation).
- n\_folds\_tune (**integer(1)**, optional)  
If **rsmp\_tune = "cv"**, number of folds used for cross-validation. If not specified by the user, default is set to 5.
- measure (NULL, named list(), optional)  
Named list containing the measures used for parameter tuning. Entries in list must either be **Measure** objects or keys to be passed to **msr()**. The names of the entries must match the learner names (see method **learner\_names()**). If set to NULL, default measures are used, i.e., "regr.mse" for continuous outcome variables and "classif.ce" for binary outcomes.

tune\_on\_folds (logical(1))

Indicates whether the tuning should be done fold-specific or globally. Default is FALSE.

*Returns:* self

**xtifedml\$summary():** Summary for DML models with IFE after calling **fit()**.

*Usage:*

```
xtifedml$summary(digits = max(3L, getOption("digits") - 3L))
```

*Arguments:*

digits (**integer(1)**)

The number of significant digits to use when printing.

**xtifedml\$confint():** Confidence intervals for DML models with IFE.

*Usage:*

```
xtifedml$confint(parm, joint = FALSE, level = 0.95)
```

*Arguments:*

parm (**numeric()** or **character()**)

A specification of which parameters are to be given confidence intervals among the variables for which inference was done, either a vector of numbers or a vector of names. If missing, all parameters are considered (default).

joint (**logical(1)**)

Indicates whether joint confidence intervals are computed. Default is FALSE.

`level (numeric(1))`

The confidence level. Default is 0.95.

*Returns:* A `matrix()` with the confidence interval(s).

`xtifedml$learner_names()`: Returns the names of the learners.

*Usage:*

`xtifedml$learner_names()`

*Returns:* `character()` with names of learners.

`xtifedml$params_names()`: Returns the names of the nuisance models with hyperparameters.

*Usage:*

`xtifedml$params_names()`

*Returns:* `character()` with names of nuisance models with hyperparameters.

`xtifedml$set_ml_nuisance_params()`: Set hyperparameters for the nuisance models of DML models with IFE.

Note that in the current implementation, either all parameters have to be set globally or all parameters have to be provided fold-specific.

*Usage:*

```
xtifedml$set_ml_nuisance_params(
  learner = NULL,
  treat_var = NULL,
  params,
  set_fold_specific = FALSE
)
```

*Arguments:*

`learner (character(1))`

The nuisance model/learner (see method `params_names`).

`treat_var (character(1))`

The treatment variable (hyperparameters can be set treatment-variable specific).

`params (named list())`

A named `list()` with estimator parameters for time-varying covariates. Parameters are used for all folds by default. Alternatively, parameters can be passed in a fold-specific way if option `fold_specific` is TRUE. In this case, the outer list needs to be of length `n_rep` and the inner list of length `n_folds_per_cluster`.

`set_fold_specific (logical(1))`

Indicates if the parameters passed in `params` should be passed in fold-specific way. Default is FALSE. If TRUE, the outer list needs to be of length `n_rep` and the inner list of length `n_folds_per_cluster`. Note that in the current implementation, either all parameters have to be set globally or all parameters have to be provided fold-specific.

*Returns:* `self`

`xtifedml$get_params()`: Get hyper-parameters for the nuisance model of `xtifedml` models.

*Usage:*

```
xtifedml$get_params(learner)
```

*Arguments:*

```
learner (character(1))
```

The nuisance model/learner (see method `params_names()`)

*Returns:* named `list()` with paramers for the nuisance model/learner.

`xtifedml$clone()`: The objects of this class are cloneable with this method.

*Usage:*

```
xtifedml$clone(deep = FALSE)
```

*Arguments:*

`deep` Whether to make a deep clone.

## See Also

Other `xtifedml`: [xtifedml\\_plr](#)

---

`xtifedml_data`

*Data Setup for Panel Data Approaches with Cluster Variables*

---

## Description

Constructs a double machine learning (DML) data backend for panel data, supporting designs with one or two cluster variables.

`xtifedml_data` objects can be initialized from a [data.table](#). The following functions can be used to create a new instance of `xtifedml_data`.

- `xtifedml_data$new()` for initialization from a `data.table`.
- [xtifedml\\_data\\_df\(\)](#) for initialization from a `data.frame`.

## Active bindings

```
all_variables (character())
```

All variables available in the data frame.

```
d_cols (character())
```

The treatment variable.

```
dbar_col (NULL, character())
```

The individual mean of the treatment variable.

```
data (data.table)
```

Data object.

```
data_model (data.table)
```

Internal data object that implements the causal panel model as specified by the user via `y_col`, `d_cols`, `x_cols`, `dbar_col`.

```
n_obs (integer(1))
```

The number of observations.

`n_treat (integer(1))`  
 The number of treatment variables.

`treat_col (character(1))`  
 "Active" treatment variable in the multiple-treatment case.

`x_cols (character())`  
 The covariates.

`y_col (character(1))`  
 The outcome variable.

`panel_id (character())`  
 The panel identifier.

`time_id (character())`  
 The time identifier.

`cluster_cols (character())`  
 The cluster variable(s).

`n_cluster_vars (integer(1))`  
 The number of cluster variables.

`approach (character(1))`  
 A character() ("fd-exact", "wg-approx" or "cre") specifying the panel data technique to apply to estimate the causal model. Default is "fd-exact".

`transformX (character(1))`  
 A character() ("no", "minmax" or "poly") specifying the type of transformation to apply to the X data. "no" does not transform the covariates X and is recommended for tree-based learners. "minmax" applies the Min-Max normalization  $x' = (x - x_{min}) / (x_{max} - x_{min})$  to the covariates and is recommended with neural networks. "poly" add polynomials up to order three and interactions between all possible combinations of two and three variables; this is recommended for Lasso. Default is "no".

## Methods

### Public methods:

- `xtifedml_data$new()`
- `xtifedml_data$print()`
- `xtifedml_data$set_data_model()`
- `xtifedml_data$clone()`

`xtifedml_data$new()`: Creates a new instance of this [R6](#) class.

#### Usage:

```
xtifedml_data$new(
  data = NULL,
  x_cols = NULL,
  y_col = NULL,
  d_cols = NULL,
  dbar_col = NULL,
  panel_id = NULL,
  time_id = NULL,
```

```

    cluster_cols = NULL,
    approach = NULL,
    transformX = NULL
)

```

*Arguments:*

data (data.table, data.frame())

Data object.

x\_cols (character())

y\_col (character(1))

The outcome variable.

d\_cols (character(1))

The treatment variable.

dbar\_col (NULL, character()) \cr Individual mean of the treatment variable (used for the CRE approach).

panel\_id (character())

The panel identifier.

time\_id (character())

The time identifier.

cluster\_cols (character())

The cluster variable(s).

approach (character(1))

A character() ("fd-exact", "wg-approx" or "cre") specifying the panel data technique to apply to estimate the causal model. Default is "fd-exact".

transformX (character(1))

A character() ("no", "minmax" or "poly") specifying the type of transformation to apply to the X data. "no" does not transform the covariates X and is recommended for tree-based learners. "minmax" applies the Min-Max normalization  $x' = (x - x_{min}) / (x_{max} - x_{min})$  to the covariates and is recommended with neural networks. "poly" add polynomials up to order three and interactions between all possible combinations of two and three variables; this is recommended for Lasso. Default is "no".

xtifedml\_data\$print(): Print xtifedml\_data objects.

*Usage:*

```
xtifedml_data$print()
```

xtifedml\_data\$set\_data\_model(): Setter function for data\_model. The function implements the causal model as specified by the user via y\_col, d\_cols, x\_cols, panel\_id, time\_id and cluster\_cols and assigns the role for the treatment variables in the multiple-treatment case.

*Usage:*

```
xtifedml_data$set_data_model(treatment_var)
```

*Arguments:*

treatment\_var (character())

Active treatment variable that will be set to treat\_col.

xtifedml\_data\$clone(): The objects of this class are cloneable with this method.

*Usage:*

```
xtifedml_data$clone(deep = FALSE)
```

*Arguments:*

deep Whether to make a deep clone.

---

|                  |                                                                                         |
|------------------|-----------------------------------------------------------------------------------------|
| xtifedml_data_df | <i>Wrapper for Double machine learning data-backend initialization from data.frame.</i> |
|------------------|-----------------------------------------------------------------------------------------|

---

**Description**

Initialization of DoubleMLData from data.frame.

**Usage**

```
xtifedml_data_df(
  df,
  x_cols = NULL,
  y_col = NULL,
  d_cols = NULL,
  panel_id = NULL,
  time_id = NULL,
  cluster_cols = NULL,
  approach = NULL,
  transformX = NULL
)
```

**Arguments**

|              |                                                                    |
|--------------|--------------------------------------------------------------------|
| df           | (data.frame())<br>Data object.                                     |
| x_cols       | (character())<br>The covariates.                                   |
| y_col        | (character(1))<br>The outcome variable.                            |
| d_cols       | (character())<br>The treatment variable(s).                        |
| panel_id     | (NULL, character())<br>The panel identifier. Default is NULL.      |
| time_id      | (NULL, character())<br>The time identifier. Default is NULL.       |
| cluster_cols | (NULL, character())<br>The cluster variables. Default is panel_id. |

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| approach   | (character(1))<br>A character() ("pooled", "pcce") specifying the panel data technique to apply to estimate the causal model. Default is "pcce".                                                                                                                                                                                                                                                                                                                                                                                       |
| transformX | (character(1))<br>A character() ("no", "minmax" or "poly") specifying the type of transformation to apply to the X data. "no" does not transform the covariates X and is recommended for tree-based learners. "minmax" applies the Min-Max normalization $x' = (x - x_{min}) / (x_{max} - x_{min})$ to the covariates and is recommended with neural networks. "poly" add polynomials up to order three and interactions between all possible combinations of two and three variables; this is recommended for Lasso. Default is "no". |

**Value**

Creates a new instance of class `xtifedml_data`.

**Examples**

```
# Generate simulated panel dataset from `xtifedml`
data = make_plpr_data(n_obs = 500, t_per = 10, dim_x = 20, theta = 1, r=2, dgp = "discontinuous")

# Set up DML data environment
x_cols = paste0("X", 1:20)

obj_xtifedml_data = xtifedml_data_df(data,
  x_cols = x_cols, y_col = "y", d_cols = "d",
  panel_id = "id",
  time_id = "time",
  cluster_cols = "id",
  approach = "pcce",
  transformX = "no")

obj_xtifedml_data$print()
```

---

|                           |                                                                                                                        |
|---------------------------|------------------------------------------------------------------------------------------------------------------------|
| <code>xtifedml_plr</code> | <i>Routine to estimate partially linear panel regression models with fixed effects within double machine learning.</i> |
|---------------------------|------------------------------------------------------------------------------------------------------------------------|

---

**Description**

Routine to estimate partially linear panel regression models with fixed effects within double machine learning.

**Format**

[R6::R6Class](#) object inheriting from [xtifedml](#).

## Details

Consider partially linear panel regression (PLR) model of form

$$Y_{it} = \theta D_{it} + l_0(X_{it}) + \lambda'_i f_t + U_{it}, D_{it} = m_0(X_{it}) + \lambda'_i f_t + V_{it}.$$

## Super class

`xtifedml` -> `xtifedml_plr`

## Methods

### Public methods:

- `xtifedml_plr$new()`
- `xtifedml_plr$set_ml_nuisance_params()`
- `xtifedml_plr$tune()`
- `xtifedml_plr$clone()`

`xtifedml_plr$new()`: Creates a new instance of this R6 class.

### Usage:

```
xtifedml_plr$new(
  data,
  ml_l,
  ml_m,
  ml_g = NULL,
  n_folds = 5,
  n_rep = 1,
  score = "orth-P0",
  dml_procedure = "dml2",
  draw_sample_splitting = TRUE,
  apply_cross_fitting = TRUE
)
```

### Arguments:

`data` (`xtifedml_data`)

The `xtifedml_data` object providing the data and specifying the variables of the causal model.

`ml_l` (`LearnerRegr`, `Learner`, `character`(1))

A learner of the class `LearnerRegr`, which is available from `mlr3` or its extension packages `mlr3learners` or `mlr3extralearners`. Alternatively, a `Learner` object with public field `task_type = "regr"` can be passed, for example of class `GraphLearner`. The learner can possibly be passed with specified parameters, for example `lrn("regr.cv_glmnet", s = "lambda.min")`.

`ml_l` refers to the nuisance function  $l_0(X) = E[Y|X]$ .

`ml_m` (`LearnerRegr`, `LearnerClassif`, `Learner`, `character`(1))

A learner of the class `LearnerRegr`, which is available from `mlr3` or its extension packages `mlr3learners` or `mlr3extralearners`. For binary treatment variables, an object of the class `LearnerClassif` can be passed, for example `lrn("classif.cv_glmnet", s = "lambda.min")`. Alternatively, a `Learner` object with public field `task_type = "regr"` or `task_type =`

"classif" can be passed, respectively, for example of class `GraphLearner`.

`ml_m` refers to the nuisance function  $m_0(X) = E[D|X]$ .

`ml_g` (`LearnerRegr`, `Learner`, `character(1)`)

A learner of the class `LearnerRegr`, which is available from `mlr3` or its extension packages `mlr3learners` or `mlr3extralearners`. Alternatively, a `Learner` object with public field `task_type = "regr"` can be passed, for example of class `GraphLearner`. The learner can possibly be passed with specified parameters, for example `lrn("regr.cv_glmnet", s = "lambda.min")`.

`ml_g` refers to the nuisance function  $g_0(X) = E[Y - D\theta_0|X]$ . Note: The learner `ml_g` is only required for the score 'IV-type'. Optionally, it can be specified and estimated for callable scores.

`n_folds` (`integer(1)`)

Number of folds. Default is 5.

`n_rep` (`integer(1)`)

Number of repetitions for the sample splitting. Default is 1.

`score` (`character(1)`)

A `character(1)` ("orth-P0" or "orth-IV"). "orth-P0" is Neyman-orthogonal score with the partialling-out formula. "orth-IV" is Neyman-orthogonal score with the IV-type formula. Default is "orth-P0".

`dml_procedure` (`character(1)`)

A `character(1)` ("dml1" or "dml2") specifying the double machine learning algorithm. Default is "dml2".

`draw_sample_splitting` (`logical(1)`)

Indicates whether the sample splitting should be drawn during initialization of the object. Default is TRUE.

`apply_cross_fitting` (`logical(1)`)

Indicates whether cross-fitting should be applied. Default is TRUE.

`xtifedml_plr$set_ml_nuisance_params()`: Set hyperparameters for the nuisance models of DML models with IFE.

*Usage:*

```
xtifedml_plr$set_ml_nuisance_params(
  learner = NULL,
  treat_var = NULL,
  params,
  set_fold_specific = FALSE
)
```

*Arguments:*

`learner` (`character(1)`)

The nuisance model/learner (see method `params_names`).

`treat_var` (`character(1)`)

The treatment variable (hyperparameters can be set treatment-variable specific).

`params` (`named list()`)

A `named list()` with estimator parameters. Parameters are used for all folds by default. Alternatively, parameters can be passed in a fold-specific way if option `fold_specific` is TRUE. In this case, the outer list needs to be of length `n_rep` and the inner list of length `n_folds`.

`set_fold_specific` (logical(1))

Indicates if the parameters passed in `params_theta` should be passed in fold-specific way. Default is FALSE. If TRUE, the outer list needs to be of length `n_rep` and the inner list of length `n_folds`.

*Returns:* self

`xtifedml_plr$tune()`: Hyperparameter-tuning within double machine learning.

The hyperparameter-tuning is performed using the tuning methods provided in the [mlr3tuning](#) package. For more information on tuning in [mlr3](#), we refer to the section on parameter tuning in the [mlr3 book](#).

*Usage:*

```
xtifedml_plr$tune(
  param_set,
  tune_settings = list(n_folds_tune = 5, rsmp_tune = mlr3::rsmp("cv", folds = 5), measure
    = NULL, terminator = mlr3tuning::trm("evals", n_evals = 20), algorithm =
    mlr3tuning::tnr("grid_search"), resolution = 5),
  tune_on_folds = FALSE
)
```

*Arguments:*

`param_set` (named list())

A named list with a parameter grid for each nuisance model/learner (see method `learner_names()`). The parameter grid must be an object of class [ParamSet](#).

`tune_settings` (named list())

A named list() with arguments passed to the hyperparameter-tuning with [mlr3tuning](#) to set up a tuning instance using `mlr3tuning::TuningInstanceBatchSingleCrit$new()` (see the [mlr3tuning](#) package).

`tune_settings` has entries

- `terminator` ([Terminator](#))

A [Terminator](#) object. Specification of terminator is required to perform tuning.

- `algorithm` ([Tuner](#) or character(1))

A [Tuner](#) object (recommended) or key passed to the respective dictionary to specify the tuning algorithm used in `tnr()`. `algorithm` is passed as an argument to `tnr()`. If `algorithm` is not specified by the users, default is set to "grid\_search". If set to "grid\_search", then additional argument "resolution" is required.

- `rsmp_tune` ([Resampling](#) or character(1))

A [Resampling](#) object (recommended) or option passed to `rsmp()` to initialize a [Resampling](#) for parameter tuning in [mlr3](#). If not specified by the user, default is set to "cv" (cross-validation).

- `n_folds_tune` (integer(1), optional)

If `rsmp_tune = "cv"`, number of folds used for cross-validation. If not specified by the user, default is set to 5.

- `measure` (NULL, named list(), optional)

Named list containing the measures used for parameter tuning. Entries in list must either be [Measure](#) objects or keys to be passed to `msr()`. The names of the entries must match the learner names (see method `learner_names()`). If set to NULL, default measures are used, i.e., "regr.mse" for continuous outcome variables and "classif.ce" for binary outcomes.

- `resolution (character(1))`  
The key passed to the respective dictionary to specify the tuning algorithm used in `tnr()`.  
`resolution` is passed as an argument to `tnr()`.

`tune_on_folds (logical(1))`

Indicates whether the tuning should be done fold-specific or globally. Default is FALSE.

*Returns:* self

`xtifedml_plr$clone()`: The objects of this class are cloneable with this method.

*Usage:*

`xtifedml_plr$clone(deep = FALSE)`

*Arguments:*

`deep` Whether to make a deep clone.

## See Also

Other xtifedml: [xtifedml](#)

## Examples

```
# An illustrative example using a regression tree (`rpart`)
library(mlr3)
library(rpart)
library(mlr3tuning)

set.seed(1234)

# Generate simulated dataset
data = make_plpr_data(n_obs = 100, t_per = 5, dim_x = 10, theta = 1,
                     r=2, rho=0.6, dgp = "discontinuous")
x_cols = paste0("X", 1:10)

# Set up DML data environment
obj_xtifedml_data = xtifedml_data_df(data,
                                     x_cols = x_cols, y_col = "y", d_cols = "d",
                                     panel_id = "id",
                                     time_id = "time",
                                     approach = "pcce")

# Set up DML estimation environment
learner = lrn("regr.rpart")
ml_l = learner$clone()
ml_m = learner$clone()

obj_xtifedml = xtifedml_plr$new(obj_xtifedml_data,
                                ml_l = ml_l, ml_m = ml_m,
                                score = "orth-P0", n_folds = 3)
param_grid = list("ml_l" = ps(cp = p_dbl(lower = 0.01, upper = 0.02),
                              maxdepth = p_int(lower = 2, upper = 10)),
                  "ml_m" = ps(cp = p_dbl(lower = 0.01, upper = 0.02),
                              maxdepth = p_int(lower = 2, upper = 10)))
```

```
tune_settings = list(n_folds_tune = 3,  
                    rsmpl_tune = mlr3::rsmpl("cv", folds = 3),  
                    terminator = mlr3tuning::trm("evals", n_evals = 5),  
                    tuner = tnr("grid_search", resolution = 10))  
  
obj_xtifedml$tune(param_set = param_grid, tune_settings = tune_settings)  
obj_xtifedml$fit()
```

# Index

- \* **xtifedml**
  - xtifedml, [3](#)
  - xtifedml\_plr, [13](#)
- data.table, [4](#), [9](#), [11](#)
- GraphLearner, [14](#), [15](#)
- Learner, [14](#), [15](#)
- LearnerClassif, [14](#)
- LearnerRegr, [14](#), [15](#)
- make\_plpr\_data, [2](#)
- Measure, [7](#), [16](#)
- msr(), [7](#), [16](#)
- ParamSet, [6](#), [16](#)
- R6, [10](#)
- R6::R6Class, [3](#), [13](#)
- Resampling, [7](#), [16](#)
- rsmp(), [7](#), [16](#)
- Terminator, [7](#), [16](#)
- tnr(), [7](#), [16](#), [17](#)
- Tuner, [7](#), [16](#)
- TuningInstance, [7](#)
- xtifedml, [3](#), [13](#), [14](#), [17](#)
- xtifedml\_data, [9](#)
- xtifedml\_data\_df, [12](#)
- xtifedml\_data\_df(), [9](#)
- xtifedml\_plr, [9](#), [13](#)